

Frontend Developer Assignment

This assignment is frontend-focused. Use any frontend framework or language you are comfortable with (e.g. React, Vue, Angular, Svelte or plain JavaScript/TypeScript); any styling approach is fine. A clean, usable UI matters, but we care far more about how your app behaves under real-world conditions: slow networks, failed requests, fast-typing users, large data, keyboard-only users and screen readers.

Choose **one** of the two problem statements below and solve it.

Problem Statement 1: Can you build a data explorer that stays fast and never shows the wrong results?

Build a searchable, filterable, sortable explorer over a large dataset (at least 10,000 records), such as orders, products or support tickets. Serve the data from a small mock API you control (e.g. MSW, json-server, or a Supabase free-tier table). The API must support server-side search, filtering, sorting and pagination, and must add random latency (200 ms to 3 s) and fail roughly 1 in 10 requests.

Your explorer must:

- **Never show stale results.** When a user types quickly or changes filters, a slow earlier response must never overwrite a newer one. Debounce search, and cancel or ignore superseded requests.
- **Keep state in the URL.** Search text, filters, sort and page (or scroll position) live in the URL. Refresh, the back/forward buttons and a shared link must all restore the exact same view.
- **Stay fast with large lists.** Scrolling through thousands of rows stays smooth. Use virtualization or infinite loading, and avoid unnecessary re-renders.
- **Handle every state honestly.** Show distinct loading, empty, error and partial-failure states. A failed request shows a clear message and a retry, and never leaves old data looking current.
- **Be accessible.** The table, filters and pagination work with keyboard alone, have visible focus, and announce result counts and errors to screen readers.
- **Include a detail view.** Clicking a row opens a detail view (route or drawer) that can be deep-linked and closed without losing the list's scroll position or filters.

Problem Statement 2: Can you build a form that never loses the user's work?

Build a multi-step form (e.g. a job application or onboarding flow with at least 3 steps, conditional fields, a repeatable section such as "add another work experience", and a file upload) backed by a free Supabase Postgres project. A saved record should also be editable later from a list view.

Your form must:

- **Autosave drafts.** Progress is saved as the user works. Closing the tab, refreshing or losing power mid-step never loses more than a few seconds of input.
- **Work offline.** If the network drops, the user can keep filling the form. Changes queue up and sync when the connection returns, with a clear indicator of what is saved and what is pending.
- **Update optimistically, and roll back honestly.** Edits appear instantly. If the server rejects a save, the UI reverts or flags the change clearly instead of silently showing data that was never stored.
- **Handle edit conflicts.** If the same record is edited in two tabs or by two people, the second save must not silently overwrite the first. Detect the conflict (e.g. with a version number or updated_at check) and let the user resolve it.
- **Validate on both sides.** Use one shared schema (e.g. Zod, Yup or JSON Schema) for client validation, and map server-side errors back to the exact fields they belong to. Validation must not block users from saving a draft.
- **Be accessible.** Every field has a proper label, errors are announced to screen readers, focus moves to the first error on submit, and the whole flow works with keyboard alone.

What to Submit

1. **Live deployment:** deploy your solution on the free tier of Vercel, Netlify or Render. We should be able to open a real running app, not just read code.
2. **Short demo video (5 minutes max):** walk through what you built and show at least one failure or edge case live, for example throttling the network in DevTools, going offline, or triggering a failed request.
3. **GitHub repo link (public):** with a README.md explaining how to run it locally, your key architecture and state-management decisions, and the tradeoffs you made.

4. **Tests:** a few meaningful automated tests (e.g. Jest or Vitest with Testing Library, Cypress or Playwright) covering the trickiest behaviour, such as race conditions, rollback or conflict handling. We value a few targeted tests over broad, shallow coverage.
5. **Sources & references:** a short list (in the README is fine) of any docs, blog posts, Stack Overflow answers, libraries or free-tier services you used; we want to see how you research.
6. **AI usage:** if you used Claude, ChatGPT, Copilot or any AI tool while building this, say so and share the chat history or export (Claude conversation share link, ChatGPT shared link, etc.) alongside your repo. We use AI tools heavily ourselves. We want to see how you direct and review AI output, not whether you used it.

All the Best!